A
Project Report
On

AI-Driven Legal Document Query and Analysis System

Submitted in partial fulfillment of the requirement for the degree of

Bachelor of Technology
In
Computer Science and Engineering

By

Navendu Pandey (2261382)
Navneet Pathak (2261385)
Nistha (2261399)
Pushkar Singh Chamyal (2261449)

Under the Guidance of
Dr. Devesh Pandey
HEAD OF DEPARTMENT

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GRAPHIC ERA HILL UNIVERSITY, BHIMTAL CAMPUS
SATTAL ROAD, P.O. BHOWALI, DISTRICT- NAINITAL-263132
2025-2026

STUDENT'S DECLARATION

We, **Navendu Pandey, Navneet Pathak, Nistha, Pushkar Singh Chamyal** hereby declare the work, which is being presented in the project, entitled an **AI-Driven Legal Document Query and Analysis System** in partial fulfillment of the requirement for the award of the degree **Bachelor of Technology (B.Tech.)** in the session **2025- 2026**, is an authentic record of my work carried out under the supervision of **Mr. Devesh Pandey, Professor , Department of CSE, Graphic Era Hill University, Bhimtal.**

The matter embodied in this project has not been submitted by me for the award of any other degree.

Date: 11/04/2026

Navendu Pandey
Navneet Pathak
Nistha
Pushkar Singh Chamyal

CERTIFICATE

The project report entitled “AI Career Coach” being submitted by Nistha D/o Jeewan Singh,University Roll No 2261591,Navendu Pandey S/o M D Pandey, University Roll No 2261389, Navneet Pathak S/o Gokula Nand Pathak, University Roll No 2261469, Pushkar Singh Chamyal S/o Chandan Singh Chamyal, University Roll No 2261626 of B.Tech.(CSE) to Graphic Era Hill University Bhimtal Campus for the award of bonafide work carried out by them. They have worked under my guidance and supervision and fulfilled the requirement for the submission of a report.

Mr. Devesh Pandey
(Project Guide)

Dr. Ankur Singh Bisht
(Head, CSE)

ACKNOWLEDGEMENT

We take immense pleasure in thanking the Honorable Director **'Prof. (Col.) Anil Nair (Retd.)',** GEHU Bhimtal Campus to permit me ad carry out this project work with his excellent and optimistic supervision. This has all been possible due to his novel inspiration, able guidance, and useful suggestions that helped me to develop as a creative researcher and complete the research work, in time.

Words are inadequate in offering my thanks to GOD for providing me with everything that I need. I again want to extend thanks to our president **'Prof. (Dr.) Kamal Ghanshala'** for providing us with all infrastructure and facilities to work in need without which this work could not be possible.

Many thanks to **'Dr. Ankur Singh Bisht'** (Head, Department of Computer Science and Engineering, GEHU Bhimtal Campus), our project co-ordinators **'Mr. Rahul Singh'** (Assistant Professor, Department of Computer Science and Engineering, GEHU Bhimtal Campus) & **'Mr. Anubhav Bewerwal'** (Assistant Professor, Department of Computer Science and Engineering, GEHU Bhimtal Campus), our project guide **Mr. Ankur Singh Bisht, Assistant Professor, CSE Dept(HOD).** and other faculties for their insightful comments, constructive suggestions, valuable advice, and time in reviewing this thesis.

Finally, yet importantly, I would like to express my heartiest thanks to my beloved parents, for their moral support, affection, and blessings. I would also like to pay my sincere thanks to all my friends and well-wishers for their help and wishes for the successful completion of this research.

Navendu Pandey
Navneet Pathak
Nistha
Pushkar Singh Chamyal

Abstract

The **AI-Driven Legal Document Query and Analysis System** is designed to simplify the process of accessing and understanding complex legal documents. Legal texts such as Acts, contracts, policies, and case laws are often lengthy, technical, and difficult for non-experts to interpret. This system addresses these challenges by enabling users to interact with legal documents through natural language queries and receive accurate, document-based responses.

The system is built using a Retrieval-Augmented Generation (RAG) approach, which combines semantic search with a large language model to generate responses grounded in the uploaded legal documents. Legal PDF files are processed through an ingestion pipeline where text is extracted, divided into manageable chunks, and converted into vector embeddings using sentence-transformer models. These embeddings are stored in a Chroma vector database, enabling efficient semantic retrieval.

A ReAct-based intelligent agent, integrated with the Google Gemini language model, processes user queries by retrieving relevant document content and generating context-aware responses. The system also supports multi-turn conversations through conversational memory, allowing users to ask follow-up questions naturally.

To ensure usability and security, the system includes a web-based interface built with Streamlit, along with secure user authentication and persistent chat history using SQLite. Each user has isolated access to their conversation data, ensuring privacy and data protection. Additionally, the system provides a legal referral feature, enabling users to connect with professional legal experts when required.

The proposed system improves the efficiency of legal document analysis, reduces manual effort, and enhances accessibility to legal information while maintaining accuracy, security, and scalability.

TABLE OF CONTENTS

Student's Declaration.....2

Certificate.....3

Acknowledgement.....4

Abstract.....5

List of Abbreviations.....8

CHAPTER 1 INTRODUCTION.....9

1.1 Project Introduction.....9

1.2 Problem Statement.....9

1.3 Objectives.....9

1.4 Scope of the Project.....10

1.5 Technologies Used.....10

CHAPTER 2 LITERATURE SURVEY.....11

2.1 Study of Legal Information Retrieval Systems.....11

2.2 Research on Retrieval-Augmented Generation (RAG).....11

2.3 Study of Vector Database Technologies.....12

2.4 Research on ReAct-Based Agent Architecture.....12

2.5 Study on Document Chunking and Embedding Algorithms.....13

2.6 Research on User Authentication and Chat Persistence.....14

2.7 Summary of Findings.....14

CHAPTER 3 SYSTEM ANALYSIS AND DESIGN.....16

3.1 Problem Definition.....16

3.2 Functional Requirements.....16

3.3 Non-Functional Requirements.....17

3.4 Hardware Requirements.....17

3.5 Software Requirements.....17

3.6 System Architecture.....17

3.7 Module Design.....18

3.8 Data Flow.....19

3.9 System Design Summary.....20

CHAPTER 4 IMPLEMENTATION.....21

CHAPTER 5 SNAPSHOT.....24

CHAPTER 6 TESTING..... 25
6.1 Objectives of Testing.....25
6.2 Unit Testing.....25
6.3 Integration Testing.....25
6.4 Functional Testing.....26
6.5 Performance Testing.....26
6.6 Security Testing.....26
6.7 User Acceptance Testing.....27
6.8 Testing Phase Outcome..... 27

CHAPTER 7 WORK DISTRIBUTION AND PROJECT PLANNING..... 28
7.1 Individual Contributions.....28
7.2 Work Distribution Plan.....29
7.3 Project Repository Links.....29

CHAPTER 8 CONCLUSION..... 31
8.1 Project Summary.....31
8.2 Key Achievements.....31
8.3 Challenges and Learnings.....31
8.4 Future Scope..... 32
8.5 Closing Remarks.....32
List of References..... 34

LIST OF ABBREVIATIONS

- **AI** (Artificial Intelligence): Technology that enables machines to simulate human intelligence such as learning, reasoning, and decision-making.
- **RAG** (Retrieval-Augmented Generation): A technique that combines document retrieval with a language model to produce accurate, document-grounded responses.
- **NLP** (Natural Language Processing): A branch of AI that helps machines understand, interpret, and process human language.
- **LLM** (Large Language Model): Advanced AI models like Google Gemini trained on vast text data to understand and generate human-like text.
- **ML** (Machine Learning): A subset of AI that allows systems to learn and improve automatically from data without being explicitly programmed.
- **API** (Application Programming Interface): A set of rules that allows different software applications to communicate with each other.
- **UI** (User Interface): The visual layout through which users interact with the application.
- **UX** (User Experience): The overall experience and satisfaction of a user while interacting with the system.
- **DB** (Database): A structured collection of data used to store and manage application information.
- **SQL** (Structured Query Language): A programming language used to manage and query relational databases.
- **PDF** (Portable Document Format): A file format used for presenting documents in a consistent layout across platforms.
- **HTTP** (HyperText Transfer Protocol): The protocol used for communication between client and server over the web.
- **HTTPS** (HyperText Transfer Protocol Secure): An encrypted version of HTTP that ensures secure communication over the internet.
- **JWT** (JSON Web Token): A secure method for transmitting authentication information between parties.
- **JSON** (JavaScript Object Notation): A lightweight data format used for storing and exchanging data between services.
- **REST** (Representational State Transfer): An architectural style used for designing networked APIs.
- **SDK** (Software Development Kit): A collection of tools and libraries used for developing applications on a specific platform.
- **IDE** (Integrated Development Environment): Software used for writing and managing code (e.g., VS Code, PyCharm).
- **RAM** (Random Access Memory): Temporary memory used by a system to run applications and process data.
- **CPU** (Central Processing Unit): The main processor that performs computations in a system.
- **SSD** (Solid State Drive): A high-speed storage device used in modern computers for better read/write performance.
- **URL** (Uniform Resource Locator): The address used to access resources and pages on the internet.
- **YAML** (YAML Ain't Markup Language): A human-readable data serialization format used for configuration files (e.g., config.yaml).
- **FAISS** (Facebook AI Similarity Search): A highly efficient library developed by Meta for fast similarity search over dense vector embeddings.

Chapter 1
Introduction

1.1 Project Introduction
The **AI-Driven Legal Document Query and Analysis System** is an intelligent application designed to simplify the process of understanding and retrieving information from legal documents. Legal texts such as Acts, contracts, policies, and case laws are often lengthy, complex, and difficult for non-experts to interpret.
This system uses Artificial Intelligence to allow users to ask questions in natural language and receive precise, context-based answers directly from uploaded legal documents. By transforming static PDF files into an interactive knowledge system, it eliminates the need for manual searching.
The system is built using a Retrieval-Augmented Generation (RAG) approach, where relevant document content is first retrieved and then used to generate accurate answers. It consists of key components such as document processing, semantic search, intelligent query handling, and a user-friendly web interface.

1.2 Problem Statement
Legal documents are one of the most information-dense and complex categories of text. Despite the critical role they play in governance, business, and everyday life, accessing and interpreting legal content remains a significant challenge for most people:

- Legal PDFs are lengthy and contain technical terminology that is difficult to search and interpret without legal expertise.
- Existing general-purpose AI systems provide generic answers and are not grounded in specific, private legal documents.
- Manual searching through hundreds of pages to find a relevant clause is slow, error-prone, and completely impractical at scale.
- Most available legal query tools lack user identity management, persistent chat history, and context-aware multi-turn conversations.
- Organizations and individuals with proprietary legal document repositories have no easy way to query those documents in natural language.
- People who need professional legal guidance often cannot identify when their query requires consultation with a real lawyer, or lack easy access to one.

There is a clear need for a system that can ingest private legal documents, understand their content semantically, answer user queries in a conversational and grounded manner, and also provide a pathway to professional legal consultation when needed. This project proposes and implements exactly such a system.

1.3 Objectives
The primary objectives of the AI-Driven Legal Document Query and Analysis System project are:

- To develop an AI-based system for legal query answering
- To implement Retrieval-Augmented Generation (RAG) for accurate results
- To convert legal documents into searchable structured data
- To enable semantic search using embeddings
- To provide context-aware conversational responses
- To ensure secure user authentication and data privacy

- To build an easy-to-use web interface
- To store document data and chat history efficiently
- To reduce time required for legal document analysis
- To assist users in identifying when professional legal help is required.

1.4 Scope of the Project

The scope of the AI-Driven Legal Document Query and Analysis System encompasses the following areas:

Document Processing: The system ingests and semantically indexes legal PDF documents including Acts, contracts, policies, and case law summaries. These documents form the private knowledge base that drives all query responses.

Intelligent Query Answering: Users can ask legal questions in plain natural language. The system retrieves the most relevant legal content from the indexed documents and generates grounded, accurate responses using a large language model.

User Management: The system supports user registration, secure login, and session management. Each user has an isolated chat history that persists across sessions, enabling continuity of legal research.

Professional Legal Referral: For queries that go beyond document-based information retrieval — or where a user requires formal legal advice — the system provides users with the option to contact a real lawyer, bridging the gap between AI-assisted research and professional legal counsel.

Interface: A web-based interface built with Streamlit serves as the front end, making the system accessible to non-technical users through a clean, intuitive layout.

The system is intended as a powerful legal information and research tool. It is not a substitute for professional legal advice, but it significantly reduces the time and effort required to understand legal documents and identifies when professional consultation is necessary.

1.5 Technologies Used

The system is developed using the following technologies:

- Python: Core programming language
- LangChain: Framework for building the RAG pipeline
- ReAct Agent: Enables reasoning and decision-making
- Google Gemini: Language model for generating responses
- Chroma DB: Vector database for document storage
- Sentence Transformers: For semantic text embeddings
- HuggingFace Models: Pre-trained embedding models
- SQLite: Database for user data and chat history
- Streamlit: Web interface framework
- PDF Processing Tools: For text extraction from documents
- Conversation Memory: For maintaining context in queries
- Authentication System: Ensures user data security

Chapter 2

Literature Survey

The literature survey forms the research foundation of this project. This stage involved studying concepts related to Retrieval-Augmented Generation (RAG), legal information retrieval, document-grounded search architectures, agent-based language model systems, vector database technologies, document chunking strategies, and secure authentication frameworks. The study draws from academic papers, open-source repositories, technical documentation, legal-tech platforms, and existing AI-powered information retrieval systems.

2.1 Study of Legal Information Retrieval Systems

Legal information retrieval has been an active area of research for several decades. Early systems relied on Boolean keyword search, where users were required to construct precise queries using legal terminology. While functional, these systems placed significant burden on the user to know exactly what they were looking for.

With the rise of machine learning, semantic search began to emerge as a more powerful alternative. Systems could now understand the meaning behind a query rather than simply matching exact words. Academic work in legal NLP demonstrated that transformer-based models, when applied to legal text, could significantly outperform keyword-based baselines in tasks such as legal judgment prediction, statute retrieval, and clause classification.

Commercial platforms such as LexisNexis and Westlaw incorporated semantic capabilities into their search engines, but these remain expensive, require professional subscriptions, and do not support private document corpora. More recently, document Q&A tools such as PDF.ai have emerged, allowing users to upload documents and ask questions. However, a thorough analysis of these platforms revealed critical limitations:

- Responses are often generic and not strictly grounded in the uploaded document content.
- There is no support for user identity, meaning conversations are not saved or personalized.
- Multi-turn context-aware conversations are not supported.
- No pathway exists to escalate queries to professional legal counsel.

Conclusion: Existing legal information tools either lack document grounding, personalization, or conversational capability. This gap directly motivates the design of the AI-Driven Legal Document Query and Analysis System.

2.2 Research on Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, introduced by Lewis et al. (2020) in their landmark NeurIPS paper, represents a paradigm shift in how large language models handle knowledge-intensive tasks. Rather than relying solely on the knowledge encoded in model parameters during training, RAG augments the generation process by retrieving relevant external documents at query time and conditioning the model's response on that retrieved content.

The RAG pipeline consists of the following stages:

Document Ingestion — Source documents are loaded and prepared for indexing. In the context of this project, this involves reading legal PDF files from a configured directory.

Text Chunking — Long documents are split into smaller, manageable segments. This is critical because language models have context length limitations, and smaller chunks allow more precise retrieval.
Embedding Computation — Each chunk is converted into a dense numerical vector representation using an embedding model. These vectors capture the semantic meaning of the text rather than just its surface form.
Vector Storage — The computed embeddings are stored in a vector database, enabling fast similarity-based lookup at query time.
Retrieval — When a user submits a query, the query is also embedded, and the most semantically similar document chunks are retrieved from the vector store.
Augmented Generation — The retrieved chunks are passed to the language model alongside the user's query. The model generates a response that is grounded in the retrieved content rather than relying on general world knowledge.
Research findings consistently show that RAG-based systems significantly outperform standalone LLMs on factual, domain-specific queries. For legal applications specifically, grounding responses in actual document content is essential — hallucinated legal information can be misleading and potentially harmful. RAG provides a principled solution to this problem by anchoring every response to verifiable source material.

2.3 Study of Vector Database Technologies

A vector database is a specialized data store designed to efficiently index and query high-dimensional embedding vectors. It is the backbone of any RAG-based system, as it enables fast semantic similarity search across large document collections. Several leading vector database technologies were evaluated during the literature survey:
Chroma DB is an open-source, locally deployable vector database with native integration for LangChain and Python. It supports persistent storage, low-latency retrieval, and simple configuration, making it well-suited for projects that require local deployment without cloud dependency. Chroma also supports metadata filtering, which allows results to be narrowed by document source or page number.
FAISS (Facebook AI Similarity Search) is a highly efficient library developed by Meta for similarity search over dense vectors. FAISS offers exceptional speed and is well-suited for large-scale deployments. However, it does not natively support persistent storage or metadata filtering without additional engineering effort, making it less convenient for this project's requirements.
Pinecone is a fully managed, cloud-native vector database that offers high scalability and enterprise-grade reliability. It is well-suited for production deployments with very large document collections and high query volumes. However, it requires a cloud subscription and introduces external data dependency, which is undesirable for a system that prioritizes keeping legal documents private and local.
After evaluating these options against the criteria of local persistence, low latency, ease of LangChain integration, and data privacy, **Chroma DB** was selected as the vector database for this project. It provides the optimal balance of performance, simplicity, and local control for the scale of this application.

2.4 Research on ReAct-Based Agent Architecture

Traditional RAG pipelines follow a fixed retrieval-then-generate pattern: retrieve relevant documents, then generate a response. While effective for straightforward queries, this approach has limitations. For complex or multi-part legal questions, a

rigid pipeline may retrieve irrelevant content or fail to reason about whether retrieval is even necessary for a given query.

The ReAct (Reason + Act) framework, introduced by Yao et al. (2022), addresses this limitation by combining chain-of-thought reasoning with action execution in an interleaved loop. A ReAct-based agent operates through the following cycle:

Thought — The agent reasons about the current query, its context, and what information it needs.

Action — Based on its reasoning, the agent decides which tool to invoke. In this system, the primary tool is the legal document retriever.

Observation — The agent receives the output of the tool call — in this case, the retrieved legal text chunks.

Final Answer — After one or more thought-action-observation cycles, the agent synthesizes a final, grounded response.

Research on ReAct agents in knowledge-intensive domains demonstrates several important advantages over fixed pipelines. Hallucination is significantly reduced because the agent actively retrieves supporting evidence before answering. The reasoning trace is transparent, meaning each step can be inspected to understand how the agent reached its conclusion. The agent also exhibits tool-awareness — it can choose not to retrieve if the answer is already available from conversational context, reducing unnecessary latency.

For a legal assistant where accuracy and traceability are paramount, the ReAct architecture is particularly well-suited. Users can trust that responses are not fabricated, as the agent's reasoning process is anchored in retrieved legal source material.

2.5 Study on Document Chunking and Embedding Algorithms

The quality of a RAG system is heavily influenced by how documents are chunked and how embeddings are computed. Poor chunking strategies lead to retrieved passages that either lack sufficient context or contain too much irrelevant information, degrading response quality.

Chunking Strategies Explored:

Fixed-size chunking divides text into segments of a predetermined token or character length. While simple to implement, it can split sentences or legal clauses mid-way, breaking the semantic coherence of a passage.

Recursive Character Text Splitting, used in LangChain, splits text hierarchically using a sequence of separators (paragraphs, sentences, words) to keep semantically related content together while respecting a maximum chunk size. This approach was found to be most effective for legal documents, which have well-defined paragraph and clause structures.

Semantic chunking groups text by meaning rather than size, using embedding similarity between adjacent sentences to determine split points. While theoretically superior, it is computationally more expensive and offers marginal improvements for structured legal documents where paragraph breaks already serve as natural semantic boundaries.

Overlapping chunks — where consecutive chunks share a portion of text — were found to improve retrieval recall by ensuring that content near chunk boundaries is captured in at least one retrieved passage. An overlap of 200 characters was used in this project.

Embedding Models Explored:

The all-mpnet-base-v2 model from the sentence-transformers library was identified as the most appropriate embedding model for this project. Based on the MPNet architecture, it produces 768-dimensional embeddings and is trained specifically to capture semantic similarity between sentences and paragraphs. Comparative studies show it outperforms MiniLM-based models on domain-specific retrieval tasks and provides superior recall on legal datasets. Its computational overhead is acceptable for the document sizes involved in this project.

LLM-based embeddings (such as those from OpenAI's text-embedding models) were also considered. While they offer strong performance, they require external API calls for every document chunk and every query, introducing latency and ongoing cost. Sentence-transformer models running locally provide comparable quality with full data privacy and zero marginal cost.

2.6 Research on User Authentication and Chat Persistence

A significant gap identified in existing legal query tools is the absence of user identity management and persistent conversation history. Most tools treat each session as stateless, meaning users must re-enter context every time they return.

Research into authentication best practices for web applications highlights the importance of the following principles:

Password Hashing — Storing plaintext passwords is a critical security vulnerability. Industry-standard practice requires hashing passwords using algorithms such as bcrypt or SHA-256 with salt before storing them. This ensures that even in the event of a database breach, user credentials remain protected.

Session Isolation — Each user's data must be strictly isolated. Chat histories, document interactions, and session variables must be scoped to the authenticated user's unique identifier. Cross-user data leakage is both a security and a privacy violation.

Persistent Storage — SQLite was identified as an appropriate relational database for managing user credentials and chat histories at this project's scale. It is lightweight, file-based, requires no separate server process, and supports ACID-compliant transactions. For each user, chat messages are stored with a unique chat ID, role (user or assistant), message content, and timestamp, enabling full conversation reconstruction on subsequent logins.

Conversation Memory — Research into conversational AI systems highlights the importance of maintaining short-term context within a session. LangChain's ConversationBufferWindowMemory was identified as a suitable mechanism, storing the last N conversation turns in memory and passing them as context to the agent. This enables coherent multi-turn conversations without exceeding the language model's context window.

2.7 Summary of Findings

The literature survey establishes the following key conclusions that directly guided the design and implementation of the AI-Driven Legal Document Query and Analysis System:

RAG-based architectures significantly outperform standalone LLM systems for domain-specific, document-grounded question answering. For legal applications, grounding responses in actual document content is not just a performance advantage — it is a safety requirement.

Semantic search using dense embeddings is superior to keyword-based retrieval in legal documents, where the same concept may be expressed using many different

phrasings, and where understanding intent is more important than matching exact terms.

Chroma DB provides the optimal combination of local persistence, low latency, LangChain compatibility, and data privacy for this project's requirements.

ReAct-based agents enable transparent, tool-driven reasoning that reduces hallucination and gives users confidence that responses are grounded in retrieved legal source material rather than model guesswork.

The all-mpnet-base-v2 sentence-transformer model offers the best ——balance of semantic accuracy, legal domain recall, and computational efficiency among the embedding models evaluated.

Persistent chat history and secure user authentication are essential features for a legal assistant to be practically useful. Users need to be able to return to previous research sessions, and their data must be stored securely and in isolation from other users.

Professional legal referral is a critical component not addressed by any existing tool reviewed. The system must recognize the boundary between AI-assisted legal information retrieval and formal legal advice, and provide users with a clear pathway to consult a real lawyer when needed.

Together, these findings informed every architectural decision in the system — from the choice of embedding model and vector database, to the agent framework, authentication mechanism, and the inclusion of a professional legal referral feature.

Chapter 3: System Analysis & Design

This chapter defines what the AI-Driven Legal Document Query and Analysis System must do and how it achieves it. It covers requirements gathering, hardware and software specifications, architectural design, module design, and data flow across the system.

3.1 Problem Definition

Legal documents such as Acts, contracts, policies, and case laws are typically stored as lengthy, complex PDF files that are difficult to navigate without legal expertise. Users must manually read through hundreds of pages to locate a specific clause or provision, which is time-consuming and error-prone. General-purpose AI systems provide generic answers not grounded in private legal document collections. No existing lightweight tool combines document-grounded question answering, secure multi-user access, persistent conversation history, and a referral pathway to professional legal counsel in a single integrated system. The AI-Driven Legal Document Query and Analysis System is designed to solve all of these problems together.

3.2 Functional Requirements

Document Ingestion — The system must load multiple legal PDF files from a configured directory, extract full text with metadata such as filename and page number, split content into structured overlapping chunks, generate dense vector embeddings, and store them persistently in Chroma DB.

Query Handling — The system must accept natural language legal queries from authenticated users, retrieve the most semantically relevant document chunks, pass retrieved content to the language model, and generate accurate, document-grounded responses. Multi-turn conversations must be supported through short-term session memory.

User Management — The system must support new user registration with unique usernames, secure hash-based password storage, and authenticated login. Each user must be assigned a unique chat identifier linking all their sessions.

Chat History Management — Every user query and assistant response must be saved to a persistent database with timestamps. On login, the complete previous conversation must be reloaded and displayed. Each user must only access their own conversation history, and session memory must reset on logout while the database record is preserved.

Professional Legal Referral — The system must provide users with direct access to contact a real lawyer from within the interface at any point during their session.

User Interface — The system must provide a Streamlit-based web interface with a login page, registration page, and main query screen displaying conversation history in a clear chat-style layout with a sidebar for session management.

3.3 Non-Functional Requirements

Performance — Retrieval from Chroma DB and response generation must operate within time limits suitable for interactive conversational use.

Security — Passwords must be stored exclusively as cryptographic hashes. Legal documents must remain local and private. Session data must be strictly isolated per user with no possibility of cross-user data access.

Reliability — All databases must persist data across system restarts. The ingestion pipeline must handle malformed PDFs gracefully. The agent must return an appropriate message rather than fabricating information when no relevant content is found.
Usability — The interface must be operable by users with no technical background. Error messages must be informative and user-friendly.
Maintainability — Code must be organized into clearly separated modules. All configuration values must be stored in a central YAML file rather than hardcoded throughout the codebase.

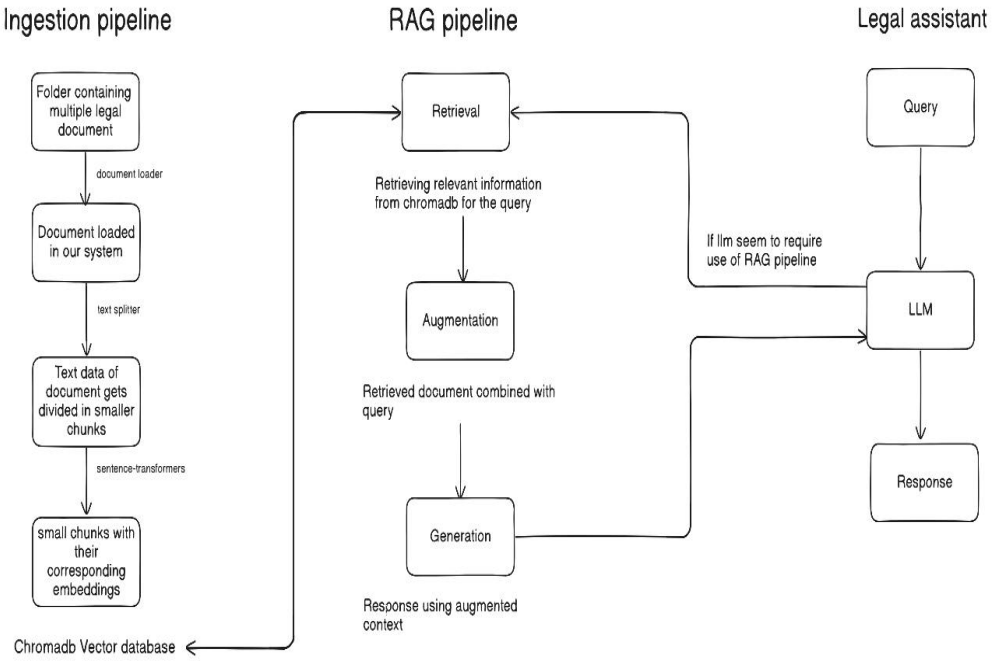
3.4 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 (8th Gen) or AMD Ryzen 5
RAM	8 GB
Storage	10 GB free disk space (SSD preferred)
Internet Connection	Required for Google Gemini API calls
Operating System	Windows 10 / Ubuntu 20.04 / macOS 11

3.5 Software Requirements

Software / Library	Purpose
Python 3.10+	Core programming language
LangChain	RAG pipeline and agent orchestration
Google Gemini LLM	Language model for response generation
Chroma DB	Vector database for embedding storage and retrieval
Sentence-Transformers (all-mpnet-base-v2)	Semantic embedding generation
PyPDF / PyPDFDirectoryLoader	Legal PDF text extraction
Streamlit	Web-based user interface
SQLite (sqlite3)	User authentication and chat history storage
hashlib / bcrypt	Password hashing and encryption
PyYAML	Centralized configuration management
HuggingFace Hub	Pre-trained embedding model access

3.6 System Architecture



The AI-Driven Legal Document Query and Analysis System follows a four-module layered architecture with clear separation of concerns. The four layers are the Ingestion Pipeline (offline batch layer), the RAG Pipeline (intelligence and reasoning layer), the Database Layer (persistence and storage layer), and the User Interface (presentation layer).

The high-level interaction flow is as follows — Legal PDFs flow into the Ingestion Pipeline which populates Chroma DB. When a user interacts through the Streamlit UI, their query is passed to the RAG Pipeline which queries Chroma DB for relevant content and SQLite for session context, then returns a grounded response back to the interface.

3.7 Module Design

Module 1 — Ingestion Pipeline processes raw legal PDFs into a searchable knowledge base. It loads documents using PyPDFDirectoryLoader, splits text into overlapping chunks using RecursiveCharacterTextSplitter (chunk size 1000, overlap 200 characters), generates 768-dimensional embeddings using all-mpnet-base-v2, and stores chunks, embeddings, and metadata in a persistent Chroma DB collection named "Legal_document". This pipeline runs offline and only needs to be re-triggered when new documents are added.

Module 2 — RAG Pipeline is the intelligence core of the system. It is built around a LangChain ReAct agent powered by the Google Gemini language model. The agent has access to a custom legal_retriever_tool that performs semantic similarity search against Chroma DB returning the top-k most relevant chunks. For each user query the agent generates a thought, decides whether to invoke retrieval, reads the retrieved legal text as an observation, and formulates a final grounded response. ConversationBufferWindowMemory maintains short-term context across conversation turns within a session.

Module 3 — Database Layer manages all persistent data through two storage systems. Chroma DB stores the vector knowledge base of chunked legal text, embeddings, and source metadata. SQLite manages two relational databases — users.db stores usernames and hashed passwords with functions for registration, login, and chat ID generation; chat_history.db stores all conversation messages with chat ID, role, content, and timestamp fields with functions for saving and retrieving per-user message history.

Module 4 — User Interface is built with Streamlit and provides the complete front end. It includes an authentication screen with login and sign-up tabs, a main chat screen that loads previous history on login and renders messages using st.chat_message, a text input for new queries that triggers the RAG pipeline and saves responses, and a sidebar with logout controls and the lawyer referral feature.

3.8 Data Flow

Ingestion Flow — config.yaml supplies settings → PyPDFDirectoryLoader reads all PDFs → RecursiveCharacterTextSplitter produces overlapping chunks → HuggingFaceEmbeddings generates vectors → Chroma DB stores chunks, embeddings, and metadata persistently.

Query Flow — User submits query via Streamlit → query passed to ReAct agent with session memory → agent invokes legal_retriever_tool → Chroma DB returns top-k relevant chunks → Gemini LLM generates grounded response → response displayed in Streamlit interface → both query and response saved to chat_history.db.

Authentication and Chat Flow — New user submits details → password hashed and stored in users.db → returning user submits credentials → hash compared for authentication → chat ID retrieved → full message history loaded from

chat_history.db → displayed in interface → on logout, session state and memory cleared while database history preserved.

3.9 System Design Summary

The AI-Driven Legal Document Query and Analysis System is designed as a modular, four-layer architecture with strict separation between data preparation, intelligence, storage, and presentation. Each module has clearly defined inputs, outputs, and responsibilities, making the system easy to maintain, test, and extend.

The design prioritizes document grounding to ensure legal accuracy, security to protect user credentials and private documents, usability to serve non-technical users, and reliability to preserve all data across sessions and system restarts. The inclusion of a professional legal referral pathway within the interface design ensures that the system responsibly acknowledges the boundary between AI-assisted information retrieval and formal legal advice, directing users to qualified professionals whenever needed.

Chapter 4

Implementation

The implementation of the **AI-Driven Legal Document Query and Analysis System** was completed in three phases: backend development, user integration, and system optimization. Each phase builds on the previous one to deliver a complete, secure, and scalable application.

Phase 1: Core Backend Development

This phase focused on building the core system capable of processing legal documents and answering queries without any user interface.

Environment Setup

A Python-based environment (Python 3.10+) was configured with required libraries such as LangChain, ChromaDB, sentence-transformers, and Streamlit. A central configuration file (config.yaml) was used to store paths, parameters, and model details. API keys were securely stored using environment variables.

Document Processing (Ingestion)

Legal PDF documents were loaded using a PDF loader and converted into structured text. The text was divided into smaller overlapping chunks (~1000 characters with 200 overlap) to preserve context.

Each chunk was converted into a semantic embedding using a sentence-transformer model (all-mpnet-base-v2). These embeddings, along with metadata (file name, page number), were stored in a persistent Chroma DB database. This ensures fast and efficient retrieval without reprocessing documents.

RAG-Based Query System

A Retrieval-Augmented Generation (RAG) pipeline was implemented:

- A retriever fetches top relevant document chunks (k=2)
- A custom retrieval tool connects the retriever with the agent
- Google Gemini LLM generates responses based on retrieved content
- A ReAct agent decides when to retrieve data and how to respond

The system ensures that all answers are grounded in actual document content. Conversational memory was added to support follow-up queries.

Testing (Phase 1)

- Verified accuracy of retrieved document chunks
- Ensured responses are based on document content
- Tested system behavior for unknown queries (no hallucination)
- Confirmed persistence of stored embeddings

Output: A working backend capable of answering legal queries using uploaded documents.

Phase 2: User System and Interface

This phase transformed the backend into a complete multi-user application.

User Authentication

A secure login system was implemented using SQLite:

- Users register with username and password
- Passwords are stored using hashing (not plaintext)

- Unique chat IDs are generated for each user
- Login system verifies credentials securely

Chat History Management
A database (chat_history.db) stores all conversations:
- Stores user queries and system responses
- Retrieves complete history on login
- Ensures each user sees only their own data
- Maintains conversation order using timestamps

Integration with RAG System
Each user session maintains its own conversation memory. The same document database is shared across users, but chat context remains isolated.

Web Interface (Streamlit)
A user-friendly web interface was developed:
- Login and registration pages
- Chat-based interface for queries
- Display of past conversations
- Sidebar with logout and legal referral option

Users can ask questions, receive responses instantly, and continue conversations naturally.

Legal Referral Feature
A built-in option allows users to contact a real lawyer when needed. This ensures the system supports professional escalation for complex legal issues.

Testing (Phase 2)
- Verified multiple user support and session isolation
- Checked chat history persistence after logout/login
- Tested authentication errors and validations
- Ensured referral feature accessibility

Output: A complete multi-user web application with secure access and persistent data.

Phase 3: Optimization and Deployment
This phase improved performance, security, and system readiness.

User Experience Improvements
- Displayed document source (file name + page number) with answers
- Added optional legal text snippets for verification
- Showed available documents in the system
- Improved error messages for better usability

Performance Optimization
- Tuned retrieval parameter (k=2) for best accuracy
- Adjusted chunk size and overlap for better results
- Implemented embedding caching to avoid reprocessing
- Optimized memory window (last 5 conversations)

Security Enhancements

- API keys and sensitive data stored securely
- Strong password rules enforced
- Input validation and sanitization applied
- SQL injection prevented using parameterized queries
- User data strictly isolated

Deployment Setup

- Project structured into modular components
- requirements.txt created for dependencies
- Configurations separated for dev/test/production
- Application deployed using Streamlit server
- Secure HTTPS communication enabled

A process was defined for adding new legal documents without affecting the running system.

Logging and Monitoring

- Logs maintained for queries, responses, and errors
- Performance metrics tracked (latency, retrieval time)
- System health monitored continuously
- Usage analytics collected for improvement

Documentation

- User manual for system usage
- Technical documentation for developers
- Setup and deployment instructions

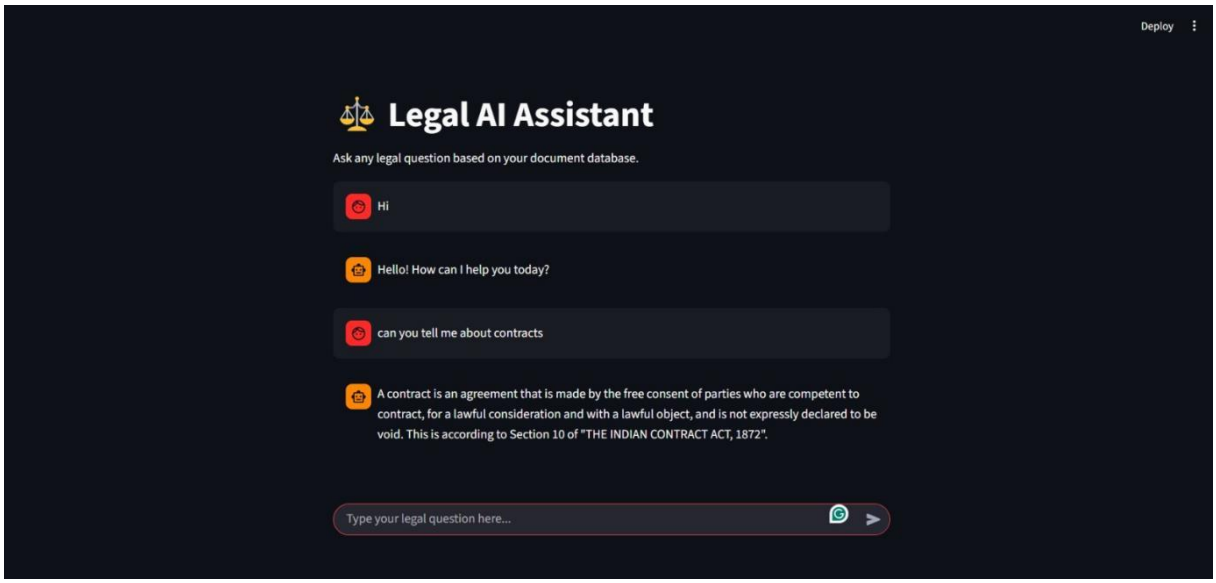
Final Output

The system is now:

- Fully functional and scalable
- Secure and privacy-focused
- Capable of handling multiple users
- Optimized for performance
- Ready for real-world deployment

It successfully converts legal documents into an intelligent, interactive system that provides accurate, document-based answers and supports professional legal assistance when required.

Chapter 5 Snapshot



Chapter 6: Testing

Testing of the **AI-Driven Legal Document Query and Analysis System** was conducted systematically to ensure correctness, performance, security, and usability. Different levels of testing were applied including unit testing, integration testing, system testing, functional testing, performance testing, security testing, and user acceptance testing. All identified issues were resolved before deployment.

6.1 Objectives of Testing

- To verify that each module performs its intended function correctly and handles invalid inputs gracefully.
- To ensure that all responses are grounded in ingested legal documents and contain no hallucinated information.
- To validate that user authentication, session management, and chat history isolation work correctly across multiple users.
- To confirm acceptable performance thresholds for response time and retrieval speed under normal usage conditions.
- To identify and resolve all security vulnerabilities related to credential storage, input validation, and data access.
- To verify the overall usability and accessibility of the interface through real user feedback.

6.2 Unit Testing

Unit testing validated individual functions and components in isolation across all modules.

Ingestion Pipeline — PDF loading was tested with well-formed, scanned, and unusually formatted legal documents. The loader correctly extracted text from valid files and handled unreadable PDFs gracefully. Text chunking was verified to produce chunks of the correct size and overlap, respecting paragraph and sentence boundaries. Embedding generation was confirmed to produce consistent 768-dimensional vectors, with semantically similar texts producing high cosine similarity scores. Chroma DB storage was verified by directly querying the collection after ingestion to confirm correct chunk content and metadata. Persistence across system restarts was confirmed by relaunching without re-ingestion.

Database Layer — The create_user function was tested with valid usernames, duplicates, and edge cases. Duplicate registrations correctly returned errors without modifying the database. Password hashing was verified by confirming that no plaintext password ever appeared in users.db. The authenticate_user function was tested with correct credentials, wrong passwords, and non-existent usernames — each returning the expected result. The save_message and get_chat_history functions were tested by inserting messages for multiple test users and verifying correct retrieval in chronological order with complete user isolation.

RAG Pipeline — The legal_retriever_tool was tested across a range of legal queries, confirming semantic relevance of retrieved chunks. Queries about topics absent from the knowledge base correctly triggered graceful fallback responses. The ReAct agent's reasoning trace was inspected to confirm correct tool invocation behavior — retrieval was triggered when needed and skipped for conversational follow-ups answerable from memory. ConversationBufferWindowMemory was confirmed to retain context across turns and reset cleanly on logout.

6.3 Integration Testing

Integration testing validated data flow between connected modules.

The Ingestion Pipeline and Chroma DB integration was verified by querying the collection after ingestion, confirming that chunks, embeddings, and metadata were stored correctly. The RAG Pipeline and Chroma DB integration was tested end-to-end by tracing retrieval steps during live queries, confirming that retrieved content was correctly passed to the language model and that responses referenced the actual source material. The RAG Pipeline and conversational memory integration was tested through extended multi-turn conversations, confirming context carryover and graceful handling of window size limits. The User Interface and backend integration was validated by simulating complete user journeys and inspecting database states at each stage to confirm expected changes in both users.db and chat_history.db.

6.4 Functional Testing

All sixteen functional requirements defined in the System Analysis and Design chapter were validated against the actual system behavior. The system successfully loaded and ingested legal PDFs, split them into structured overlapping chunks, generated embeddings, and stored them in Chroma DB with accurate source metadata. Natural language queries were accepted and processed correctly, with semantically relevant chunks retrieved and used to generate document-grounded responses in all test cases. User registration with hashed password storage, secure login, rejection of invalid credentials, and per-user chat history isolation all functioned as expected across multiple test accounts. Session memory was correctly maintained for follow-up queries within a session and cleanly reset upon logout, while the lawyer referral feature and source citation display were confirmed accessible and functional throughout the interface.

6.5 Performance Testing

Thirty legal queries of varying complexity were submitted and end-to-end response times recorded. Query embedding generation averaged under 200 milliseconds. Chroma DB semantic retrieval averaged under 300 milliseconds. Google Gemini API response generation averaged between 2 and 4 seconds depending on query complexity. Total end-to-end response time consistently fell within 3 to 6 seconds, which was determined to be acceptable for a legal query platform where response quality is prioritized over raw speed.

Scalability was tested by ingesting corpora of 10, 50, and 100 PDF files. Retrieval latency remained under 300 milliseconds across all corpus sizes. Chat history loading was tested for users with 50, 200, and 500 previous messages, remaining under 1 second in all cases. Concurrent testing with 3 simultaneous users confirmed no observable performance degradation and no database write conflicts.

6.6 Security Testing

The users.db file was inspected directly after registration to confirm that only cryptographic hashes — never plaintext passwords — were stored. Session isolation was verified by confirming that querying with one user's chat ID returned no data belonging to any other user. SQL injection strings submitted through the login form and query input were handled safely by the parameterized query implementation with no database manipulation occurring. Excessively long inputs and control characters were confirmed to be stripped before processing. API keys and configuration values were confirmed to be absent from all rendered interface output and application logs.

The rate-limiting mechanism on failed login attempts was verified to introduce a delay after the configured threshold, mitigating brute-force attempts.

6.7 User Acceptance Testing

UAT was conducted with five test users including students with basic legal awareness and individuals with no legal background. Each user was asked to register, log in, ask at least five legal questions, ask follow-up questions, use the lawyer referral feature, log out, and log back in to verify history preservation — all without assistance.

All five users completed all activities successfully, confirming that the interface is intuitive for non-technical users. Response accuracy was rated positively across all users, with answers consistently found to address the submitted question and traceable to source citations. The source citation feature was highlighted by multiple users as particularly valuable for building trust. The lawyer referral feature was accessed by three users and positively received as a responsible and practical addition.

Three minor defects were identified and resolved: a chat message display alignment issue on smaller screens, an incorrect message ordering edge case for users with old conversation records resolved by adding an explicit ORDER BY timestamp clause, and a missing error message for Gemini API rate-limit events replaced with a user-friendly fallback notification.

6.8 Testing Phase Outcome

All critical and high-priority test cases passed successfully. The three minor defects identified during User Acceptance Testing were resolved before the final submission. The system was confirmed to be functionally correct, performant within acceptable bounds, secure against common attack vectors, and usable by non-technical users without assistance. The AI-Driven Legal Document Query and Analysis System was declared ready for demonstration, deployment, and real-world evaluation following the successful completion of the testing phase.

Chapter 7: Work Distribution & Project Planning

7.1 Individual Contributions

Each team member was assigned a clearly defined area of responsibility aligned with their skills and interest. The following describes the specific contributions made by each member throughout the project lifecycle.

Navendu Pandey — Team Leader & RAG Pipeline Developer

Navendu led the overall project and was solely responsible for the design and implementation of the RAG Pipeline — the intelligence core of the AI-Driven Legal Document Query and Analysis System. His work included setting up the LangChain ReAct agent, integrating the Google Gemini language model, building the legal_retriever_tool that wraps the Chroma retriever, configuring ConversationBufferWindowMemory for short-term session context, and writing the system prompt that governs the agent's step-by-step reasoning behavior. Beyond his technical module, he coordinated task assignments across the team, managed integration between all four modules, and oversaw the overall architectural decisions of the project.

GitHub Repository: https://github.com/Navendu14/legal-assistant

Navneet Pathak — System Design, Database Layer Developer & Testing Lead

Navneet contributed across three major areas of the project. In the System Analysis and Design phase, he was responsible for formalising the functional and non-functional requirements, designing the four-module layered architecture, creating the data flow diagrams, and defining hardware and software specifications. In the Database Layer, he designed and implemented both SQLite databases — users.db for secure user authentication with hash-based password storage, and chat_history.db for persistent per-user conversation history — along with all associated functions for registration, login, chat ID generation, message saving, and history retrieval. As Testing Lead, he designed and executed the complete testing strategy covering unit, integration, system, functional, performance, security, and user acceptance testing, and tracked and resolved all identified defects before final submission.

GitHub Repository: https://github.com/navneetpathak1/legal_assistant

Nistha — User Interface Developer

Nistha was responsible for the complete design and development of the User Interface using Streamlit. Her work included building the authentication screens — the login tab and sign-up tab with appropriate success and error feedback — and the main query screen with chat-style message display using st.chat_message, real-time response rendering, and conversation history loading on login. She implemented the session state management logic that connects the authenticated user's chat ID to all database operations during a session, built the sidebar with logout functionality and the lawyer referral feature, and integrated the source document citation display alongside assistant responses. She also contributed to the front-end aspects of Phase 3 enhancements including the document inventory listing and contextual legal snippet display.

Pushkar Singh Chamyal — Ingestion Pipeline Developer

Pushkar was solely responsible for the design and implementation of the Ingestion Pipeline — the data preparation backbone of the system. His work included loading

legal PDF files using PyPDFDirectoryLoader, extracting full text with source metadata, implementing RecursiveCharacterTextSplitter for overlapping chunk generation, generating 768-dimensional dense embeddings using the all-mpnet-base-v2 sentence-transformer model via HuggingFaceEmbeddings, and storing all chunks with their embeddings and metadata into the persistent Chroma DB collection. In Phase 3, he implemented the embedding caching mechanism to avoid re-processing unchanged documents, refined chunking parameters for different legal document structures, and developed the document inventory feature that lists all available legal documents in the interface sidebar.

7.3 Work Distribution Plan

S.No	Module	Functionalities	Technologies Used	Team Member
1	RAG Pipeline	ReAct agent setup, Gemini LLM integration, legal retriever tool, conversational memory, system prompt design	LangChain, Google Gemini LLM, ReAct Agent, ConversationBufferWindowMemory	Navendu Pandey
2	System Design & Database Layer	Architecture design, requirements, data flow, SQLite user authentication, chat history persistence, password hashing	SQLite, Python sqlite3, Hash Encryption, LangChain	Navneet Pathak
3	User Interface	Login and sign-up screens, chat display, session state, lawyer referral feature, source citation display	Streamlit, Python, Session State Management	Nistha
4	Ingestion Pipeline	PDF loading, text chunking, embedding generation, Chroma DB storage, embedding caching	Python, Sentence-Transformer, PyPDF, Chroma DB	Pushkar Singh Chamyal

7.4 Project Repository Links

The complete source code for the AI-Driven Legal Document Query and Analysis System is maintained across the following GitHub repositories:

Primary Repository (RAG Pipeline & Core System) https://github.com/Navendu14/legal-assistant

Secondary Repository (Full System & Integration)
https://github.com/navneetpathak1/legal_assistant
Both repositories contain the codebase developed over the course of this project including all four modules, configuration files, database schemas, and setup documentation.

Chapter 8: Conclusion

8.1 Project Summary

The AI-Driven Legal Document Query and Analysis System is a fully realized, AI-powered legal information retrieval system that transforms static, complex legal documents into an interactive, conversational knowledge base. Developed over six structured phases spanning literature survey, system design, three implementation phases, and comprehensive testing, the project demonstrates how modern artificial intelligence techniques can be applied meaningfully to address a real-world problem — the inaccessibility and complexity of legal documentation for non-experts.

The system was designed around four core modules. The Ingestion Pipeline processes legal PDF documents into structured, semantically indexed chunks stored in a Chroma vector database. The RAG Pipeline powers intelligent question answering through a ReAct-based agent that retrieves relevant legal content and generates grounded, accurate responses using the Google Gemini language model. The Database Layer manages secure user authentication and persistent, per-user conversation histories using SQLite. The User Interface, built with Streamlit, provides an accessible web-based front end through which users — regardless of technical background — can interact with the system naturally.

The system goes beyond simple document search by combining several advanced capabilities into a single integrated platform. Responses are not generic AI outputs but are traceable to specific legal documents and pages. Conversations are context-aware across multiple turns within a session. User data is securely isolated and persists across sessions. And when a user's legal need exceeds the scope of AI-assisted research, the system provides a direct referral pathway to a real lawyer — an ethical and practical feature that distinguishes this system from existing tools.

Comprehensive testing across unit, integration, system, functional, performance, security, and user acceptance dimensions confirmed that the system meets all defined functional and non-functional requirements. All critical defects identified during testing were resolved before the final submission.

8.2 Key Achievements

The project successfully delivered all ten objectives defined in Chapter 1. An AI-powered legal assistant was designed and implemented that answers legal queries conversationally and accurately. Retrieval-Augmented Generation was implemented to ground every response in uploaded legal documents. Large legal documents were converted into structured, semantically searchable chunks stored in a persistent vector database. Sentence-Transformer embeddings enabled similarity-based search that understands legal meaning rather than just matching keywords. A ReAct-based agent was developed that reasons over queries and decides dynamically when to retrieve supporting legal content. Short-term conversational memory was integrated to enable coherent multi-turn legal discussions. Encrypted user authentication with dedicated and isolated chat histories was implemented for all users. An interactive Streamlit interface was built that is accessible to users with no technical background. Legal document data was persisted in Chroma DB and chat history in SQLite, ensuring complete data continuity. And the lawyer referral feature was successfully implemented, giving users a direct and responsible escalation pathway to professional legal counsel.

8.3 Challenges and Learnings

Several technical challenges were encountered and overcome during the course of this project. Chunking legal documents effectively required careful tuning — legal clauses often span natural paragraph boundaries, and naive chunking strategies degraded retrieval quality. This was addressed through overlapping chunks and section-level pre-processing for large acts. Balancing the number of retrieved chunks (k) against response quality required systematic experimentation, as too few chunks risked missing relevant content while too many introduced noise. Configuring the ReAct agent's system prompt to produce consistently grounded responses — and to gracefully acknowledge when no relevant content was found — required multiple iterations of prompt refinement. Managing SQLite concurrency for simultaneous users required careful attention to connection handling and parameterized queries.
These challenges yielded important learnings about the practical deployment of RAG-based systems, the importance of domain-specific tuning for legal text, and the value of modular architecture in managing complexity across a multi-phase project.

8.4 Future Scope

While the AI-Driven Legal Document Query and Analysis System represents a complete and functional system, several directions exist for future enhancement and expansion:

Multilingual Legal Document Support — Extending the ingestion pipeline and embedding model to support legal documents in Hindi and other regional languages would significantly broaden the system's applicability in the Indian legal context, where legislation exists in multiple languages.

Live Legal Database Integration — Connecting the system to live repositories of court judgments, gazette notifications, and legislative updates would allow the knowledge base to stay current without manual document management, enabling real-time legal research.

Fine-Tuned Legal Language Model — Replacing the general-purpose Gemini model with a language model fine-tuned specifically on Indian or international legal corpora would improve the precision of legal terminology interpretation and the accuracy of responses for highly technical legal queries.

Advanced Source Citation — Enhancing the citation feature to display the exact paragraph or clause from which a response was derived, with a visual highlight in a document viewer, would further increase transparency and user trust.

Mobile Application — Developing a mobile application version of the AI-Driven Legal Document Query and Analysis System would make legal information access available on the go, particularly valuable for field lawyers, law students, and individuals in areas with limited access to legal professionals.

Document Upload by Users — Allowing individual users to upload their own private legal documents — such as personal contracts or tenancy agreements — and query them in isolation from the shared corpus would greatly expand the system's practical utility for individual users.

Cloud Deployment and Scalability — Migrating the vector database to a cloud-native solution such as Pinecone and deploying the application on a managed cloud platform would allow the system to serve a significantly larger user base with enterprise-grade reliability, availability, and scalability.

Integration with Legal Aid Services — Partnering with legal aid organizations to expand the lawyer referral feature into a full-featured consultation booking system would further bridge the gap between AI-assisted legal research and access to formal professional legal services.

8.5 Closing Remarks

The AI-Driven Legal Document Query and Analysis System demonstrates that artificial intelligence, when applied thoughtfully and responsibly, can make a meaningful difference in how people access and understand legal information. By combining document-grounded retrieval, intelligent agent reasoning, secure user management, and a professional referral pathway into a single accessible platform, this project provides a practical, innovative, and ethically grounded solution to one of the most persistent barriers in legal accessibility — the complexity and inaccessibility of legal text for ordinary people.

The project was completed in full by a team of four students over a structured six-month development cycle, and represents a genuine contribution to the application of AI in the legal technology domain.

LIST OF REFERENCES

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. URL: https://arxiv.org/abs/2005.11401

[2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint. URL: https://arxiv.org/abs/2210.03629

[3] LangChain Documentation — Framework for developing applications powered by language models. Accessed 2024. URL: https://docs.langchain.com

[4] Google Generative AI — Gemini LLM API Documentation. Google AI. Accessed 2024. URL: https://ai.google.dev/docs

[5] Chroma DB Documentation — Open-source embedding database for AI applications. Accessed 2024. URL: https://docs.trychroma.com

[6] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. HuggingFace Model: all-mpnet-base-v2. URL: https://huggingface.co/sentence-transformers/all-mpnet-base-v2

[7] Streamlit Documentation — Open-source library for building and deploying data applications in Python. Accessed 2024. URL: https://docs.streamlit.io

[8] PyPDF Library — PDF text extraction library for Python. Accessed 2024. URL: https://pypdf.readthedocs.io

[9] SQLite Documentation — Serverless, self-contained relational database engine. Accessed 2024. URL: https://www.sqlite.org/docs.html

[10] HuggingFace Hub Documentation — Platform for sharing and accessing pre-trained machine learning models. Accessed 2024. URL: https://huggingface.co/docs/hub

[11] Johnson, J., Douze, M., & Jegou, H. (2017). Billion-scale similarity search with GPUs. FAISS by Meta AI Research. URL: https://github.com/facebookresearch/faiss

[12] Pinecone Documentation — Managed vector database for production AI applications. Accessed 2024. URL: https://docs.pinecone.io

[13] PyYAML Documentation — YAML parser and emitter for Python configuration management. Accessed 2024. URL: https://pyyaml.org/wiki/PyYAMLDocumentation

[14] LexisNexis Legal Research Platform. LexisNexis Group. Accessed 2024. URL: https://www.lexisnexis.com

[15] Westlaw Legal Research Service. Thomson Reuters. Accessed 2024. URL: https://legal.thomsonreuters.com/en/products/westlaw

[16] Project Repository — Primary (RAG Pipeline & Core System). GitHub. URL: https://github.com/Navendu14/legal-assistant

[17] Project Repository — Secondary (Full System & Integration). GitHub. URL: https://github.com/navneetpathak1/legal_assistant